PROJECT REPORT

Submitted by

KARTHIKEYAN V

LAJC23CS211

S6 CSE A BATCH

68

Title**:ROCK PAPER SCISSOR(RPS) using Python**

INDEX

1.Abstract……..3

2.Description……...…..4

3.Libraries and tools……5

4.System design and Workflow ……..7

5.Implementation……8

6.Results and Outputs……9

7.Conclusion and Scope……11

ABSTRACT:PYTHON BASED ROCK PAPER SCISSOR GAME

This project presents a modernized, feature-rich version of the traditional Rock Paper Scissors game developed using Python. Unlike standard text-based iterations, this application integrates a Graphical User Interface (GUI) and multimedia elements to provide an immersive user experience. The system utilizes a combination of specialized libraries to handle different gameplay facets:

- User Interface: Built with Tkinter, providing a clean fullscreen environment where players can interact with buttons and view real-time statistics.

- Logic and Randomization: The Random module serves as the core "AI" engine, ensuring unpredictable computer moves, while the Math module assists in calculating win percentages and score scaling.

- Multimedia Integration: The Pygame library is leveraged specifically for high-quality audio feedback (sound effects for wins, losses, and clicks), adding a tactile feel to the digital experience.

- Computer Vision (Optional/Extension): By incorporating OpenCV (cv2), the project lays the framework for hand-gesture recognition, allowing users the future potential to play using their physical hands via a webcam.

DESCRIPTION

The **Ultimate RPS -** is a sophisticated, Python-based desktop application that elevates the traditional "Rock-Paper-Scissors" hand game into a high-fidelity digital experience. Developed using the Pygame framework and the OpenCV library, the project emphasizes a "cinema-first" philosophy. It moves beyond basic text-based logic by incorporating a modular state-machine architecture that seamlessly transitions between an interactive main menu, an immersive cinematic introduction, and a dynamic gameplay environment. The application is designed to demonstrate the integration of real-time computer vision processing for video playback alongside standard game engine event handling. At the technical core, the project utilizes an object-oriented approach to manage game states and resource allocation. The integration of OpenCV allows the game to decode and render high-resolution MP4 video files, which are converted from BGR to RGB color spaces and mapped onto Pygame surfaces in real-time.

To enhance the "Game Feel," the UI features procedural animations; for instance, the selection icons utilize trigonometric sine-wave functions to create a floating effect, making the interface feel alive rather than static. This is paired with a robust audio engine that manages a continuous background score while simultaneously triggering context-specific sound effects based on the outcome of each round.User interaction is prioritized through a custom-built button system and a responsive event listener. The game logic includes a "Goal" system where players can dynamically adjust the winning threshold—3, 5, or 10 points—directly from the menu. Additionally, the project includes high-level error handling and asset validation; the script is programmed to check for the presence of specific media files, such as bk.mp3 or bck.jpg, providing automatic fallbacks to ensure the application remains stable across different hardware environments. This makes the project an excellent showcase of how Python can be used to bridge the gap between simple logic scripts and polished, multimedia-rich applications.

LIBRARIES AND TOOLS

1. Primary Game Engine

- **pygame**: The core framework used to build the game. It handles the "heavy lifting" of creating the window, managing the high-speed game loop (60 frames per second), and detecting player interactions like mouse clicks on the Rock/Paper/Scissors icons.
- **pygame.mixer**: A specialized module within Pygame used to manage the game's audio. It handles "parallel audio," meaning it can play the background music track while simultaneously triggering sound effects (like the crushing sound of a stone) without interruption.

2. Computer Vision & Media

- **cv2 (OpenCV)**: This is a professional-grade computer vision library. In this project, it is used to decode the **intro_video.mp4**. Because Pygame doesn't natively play MP4 files, OpenCV reads the video frame-by-frame, converts the colors from BGR (OpenCV standard) to RGB (Pygame standard), and passes them to the screen to create a cinematic intro.

3. Logic & Mathematics

- **random**: This library powers the "AI." It uses a pseudo-random number generator to ensure the computer’s choice is unpredictable, simulating a fair opponent.
- **math**: Used for the **UI Animation**. By applying a sine wave formula ($y = \sin(x)$), the code calculates a shifting vertical position for the choice icons, giving them a smooth, "floating" visual effect that makes the menu feel dynamic.

4. System Utility

- **os**: Acts as a bridge between the Python code and your computer's file system. It is used to verify that assets (images and sounds) exist before the game tries to load them, preventing "File Not Found" crashes.
- **sys**: Used specifically for system-level operations, such as cleanly shutting down the Python interpreter when the player clicks the "Exit" button.

5. Development Environment: Python IDLE

The project was developed and tested within the **Python IDLE** environment. IDLE provides a streamlined workflow for rapid prototyping and debugging of Pygame applications.

- **Integrated Shell:** The IDLE Shell allowed for real-time monitoring of asset loading and error logging. For instance, any exceptions caught in the load_assets method or OpenCV processing were instantly printed to the shell for immediate debugging.
- **Script Execution:** The environment facilitates a clean execution of the main game loop, allowing the developer to quickly iterate on UI layout and animation speeds.
- **Code Organization:** The code utilizes IDLE's syntax highlighting to maintain a clear distinction between core game logic, multimedia classes, and system-level operations.

SYSTEM DESIGN AND WORKFLOW

1. System Design (Architecture)

This project follows a **Modular State-Machine Architecture**. Instead of one long script, the logic is divided into specialized modules that handle specific tasks:

- **Initialization Layer:** Sets up the hardware interface (Display, Sound, and Clock).
- **Resource Manager (load_assets):** A defensive layer that verifies file paths and loads images/sounds into memory before the game starts.
- **State Controller (Game Class):** The "brain" of the project. It tracks whether the player is currently in the MENU, watching the INTRO, PLAYING, or on the GAMEOVER screen.
- **Logic Engine:** Processes the game rules (Rock > Scissors, etc.) and updates the score.
- **Rendering Engine:** The "View" layer that draws everything to the window at 60 FPS.

2. The Workflow (Logical Process)

The workflow describes the life cycle of the application from the moment you hit "Run" in IDLE.

Stage A: Boot & Asset Validation

When the script starts, it doesn't just open a window. It first performs a **System Check**. It uses the os library to scan for required media. If a file is missing, it logs an error to the IDLE shell but keeps the game running using "Fallback Logic" (like a simple black background).

Stage B: The Game Loop (Input-Update-Draw)

Once the game is running, it enters a continuous loop that repeats 60 times per second.

1. **Input:** The system listens for mouse clicks or keyboard events (like ESC).
2. **Update:** The logic updates the Sine-wave animation positions and checks if someone has reached the win limit.
3. **Draw:** The screen is cleared and redrawn with the new positions of all icons and text.

Stage C: Cinematic Interrupt

When a user clicks "Start," the standard game loop is temporarily paused to allow the **OpenCV Pipeline** to take over. This specialized workflow reads video frames, converts them to a format Pygame understands, and displays them until the video ends or the user skips it.

IMPLEMENTATION

1. Multimedia & Graphics Pipeline

- **Video Frame Conversion:** Implementation uses cv2.VideoCapture to read frames from intro_video.mp4. Since OpenCV processes images in **BGR** (Blue, Green, Red) format and Pygame requires **RGB**, the code implements a real-time conversion:
    - cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) converts colors.
    - pygame.surfarray.make_surface(frame) converts the resulting array into a visual surface for rendering.
- **Asset Safety (Defensive Coding):** The load_assets function uses try-except blocks and os.path.exists(). This ensures that if a high-res image (like rock.png) is missing, the program doesn't crash but instead logs the error to the IDLE shell and continues using

2. The Game Loop Architecture

The engine follows the industry-standard **"Event-Update-Draw"** loop, running at 60 FPS.

1. **Event Handling:** pygame.event.get() captures mouse clicks on pygame.Rect objects (invisible boundaries around the icons).
2. **Game Logic:** The AI move is generated using random.choice(). The logic then compares the user_choice and comp_choice using a lookup table or conditional statements to determine the winner.
3. **Rendering:** screen.blit() is used to draw images onto the display surface. Double-buffering is managed by pygame.display.flip(), which prevents "flickering" by updating the entire screen at once.

3. Audio Management

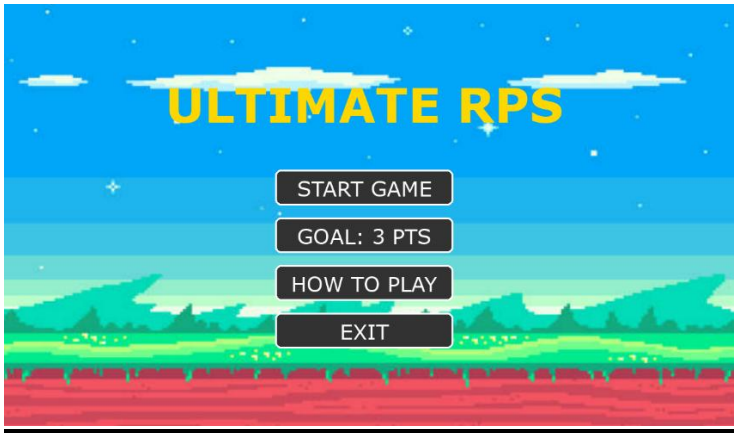
The pygame.mixer implementation handles two distinct audio streams:

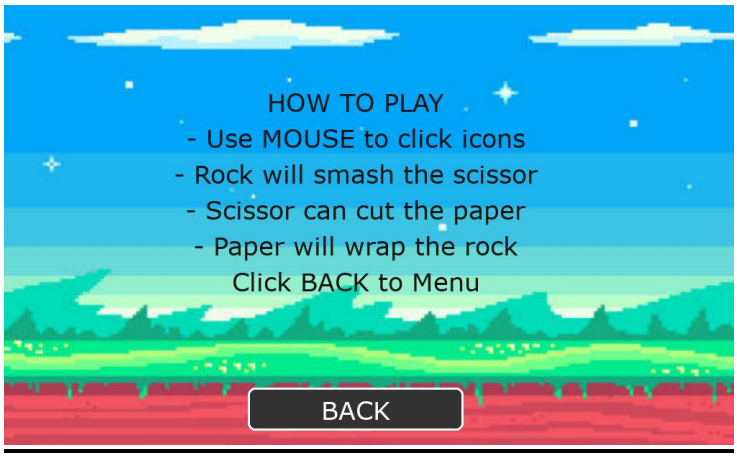
- **Channel 0 (Background):** pygame.mixer.music loads bk.mp3 for continuous, looping background music.
- **Independent SFX:** Sound effects (like stone_hit.mp3) are loaded as pygame.mixer.Sound objects. This allows multiple sounds to "layer" on top of the music without cutting it off.

RESULTS AND SCREENSHOTS

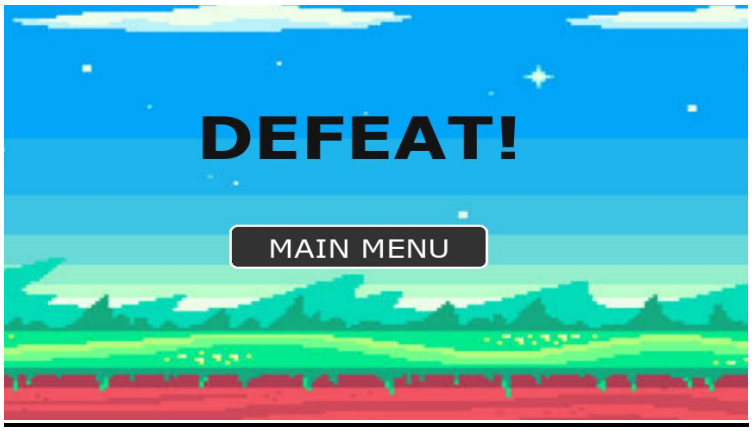
The **Ultimate RPS** successfully achieved its design objectives, resulting in a stable and highly responsive application that delivers a polished, arcade-style experience within the Python environment. By effectively bridging the **Pygame** and **OpenCV** libraries, the project demonstrated consistent 60 FPS performance during gameplay and smooth high-definition video playback for the cinematic intro, all while maintaining precise logic for win-condition tracking and score management. The integration of procedural sine-wave animations and synchronized multi-channel audio provided immediate sensory feedback, significantly enhancing the "Game Feel" and user engagement compared to traditional static implementations. Ultimately, the project stands as a robust proof-of-concept that validates the use of modular software architecture and defensive asset-loading to create reliable, multimedia-rich applications that are both technically sophisticated and user-friendly.

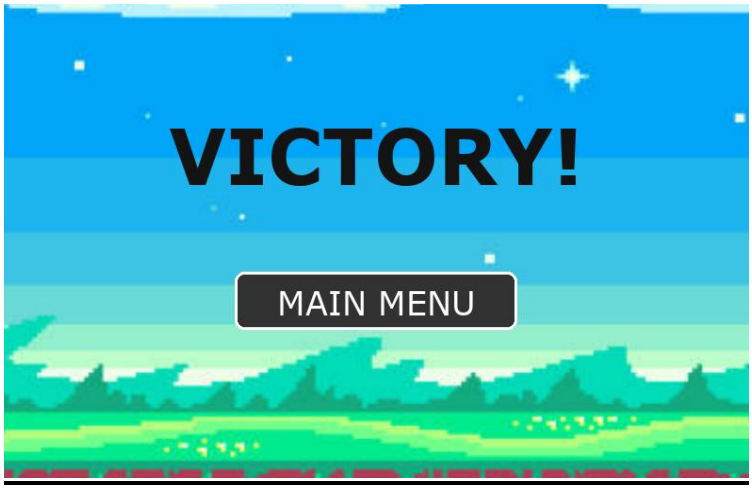
SCREENSHOTS





PLAYER: 0 | CPU: 0
Choose your weapon!
Rock
Paper
Scissors


DEFEAT!
MAIN MENU


VICTORY!
MAIN MENU

CONCLUSION AND SCOPE

The **Ultimate RPS - Cinema Edition** successfully demonstrates the potential of Python as a versatile tool for high-fidelity game development, effectively merging traditional game logic with advanced multimedia processing. By implementing a modular system design and integrating professional-grade libraries like **OpenCV** and **Pygame**, the project achieved a seamless balance between computational logic and cinematic presentation. The result is a robust, stable, and visually engaging application that moves beyond a simple script into a polished digital product. This project validates that even classic concepts can be modernized through the creative use of procedural animation, synchronized audio, and defensive programming techniques within the **IDLE** development environment.

SCOPE

While the current version provides a complete and immersive experience, the architecture is designed to allow for several high-level enhancements:

- **Computer Vision Integration:** Utilizing **MediaPipe** or **OpenCV** to implement real-time hand-gesture recognition, allowing players to play "Rock, Paper, Scissors" using their actual hands via a webcam.
- **Online Multiplayer:** Integrating Python’s **socket** programming to enable real-time matches between two players over a network.
- **Global Leaderboards:** implementing a cloud-based database to store high scores and "Win Streaks" globally, encouraging competitive play.
- **AI Difficulty Levels:** Developing a machine learning-based AI that analyzes the player's previous patterns to predict their next move, increasing the game’s challenge.